

Universidade Federal de Minas Gerais
Escola de Engenharia
Curso de Graduação em Engenharia de Sistemas

Desenvolvimento de um Algoritmo Heurístico para Otimização de Leitos Hospitalares

Gustavo Ferreira Tavares

Orientadoras:
Profa. Gabriela Nunes Lopes
Profa. Luiza Bernardes Real

Belo Horizonte, março de 2026

Capítulo 1

Introdução

Hospitais são instituições fundamentais para a manutenção da saúde pública, enfrentando uma demanda crescente por seus serviços, impulsionada por fatores como o aumento e o envelhecimento populacional [1, 2]. Contudo, essa demanda ocorre em um cenário de alta pressão por eficiência, devido aos custos ascendentes dos serviços de saúde e à limitação de recursos financeiros e físicos [1, 2]. Nesse contexto, a gestão eficiente dos recursos hospitalares não é apenas uma meta operacional, mas uma necessidade estratégica para garantir a sustentabilidade do sistema e a qualidade do atendimento ao paciente [1, 3].

No Brasil, onde o Sistema Único de Saúde (SUS) enfrenta desafios históricos de superlotação e subfinanciamento, a otimização de leitos torna-se ainda mais crucial. Um dos recursos mais críticos e, frequentemente, o gargalo das operações hospitalares, é a disponibilidade de leitos de internação [1, 3]. A gestão ineficiente de leitos gera impactos em cascata por todo o hospital, sendo apontada como uma das principais causas da superlotação nos serviços de emergência, um fenômeno conhecido como bed-block ou "bloqueio de leitos" [1, 3]. Essa superlotação resulta em longos tempos de espera para pacientes admitidos, desvio de ambulâncias para outras unidades e, conseqüentemente, compromete a segurança e a qualidade do cuidado [1, 3].

A complexidade do problema é acentuada pela natureza estocástica do ambiente hospitalar, marcada pela incerteza nas chegadas de pacientes, especialmente os de emergência, e pela variabilidade no tempo de permanência (LOS - Length of Stay) de cada paciente [4, 3, 2]. Além disso, a alocação de pacientes a leitos transcende uma simples otimização de ocupação. Trata-se de um desafio multiobjetivo que deve conciliar interesses conflitantes de múltiplos stakeholders: pacientes, profissionais da saúde e a administração hospitalar [2].

Do ponto de vista do paciente, busca-se conforto e adequação clínica, como a alocação em quartos com companheiros de faixa etária similar e a minimização de transferências internas [2]. Para os profissionais da saúde, a proximidade geográfica de seus pacientes otimiza as visitas e o tempo de deslocamento, sendo crucial um equilíbrio na carga de trabalho para evitar a sobrecarga e garantir a qualidade da assistência [2]. A administração, por sua vez, visa maximizar a receita e a utilização dos recursos, sem negligenciar objetivos sociais como a equidade no acesso ao tratamento para diferentes perfis de pacientes [5, 6].

Para superar os métodos tradicionais, diversas abordagens de Pesquisa Operacional, como programação matemática, simulação e modelos baseados em teoria das filas, têm sido propostas para enfrentar esse desafio [1, 3]. Métricas como taxa de ocupação, intervalo de substituição entre pacientes (turnaround time) e tempo médio de permanência (length of stay)

são frequentemente utilizadas na literatura para avaliar a eficiência da alocação de leitos [3]. No entanto, o problema de alocação de leitos (PBA - Patient-Bed Allocation ou PAS - Patient Admission Scheduling) é reconhecido por sua alta complexidade computacional, sendo frequentemente classificado como NP-difícil ou NP-completo [4, 6, 2]. Isso significa que encontrar uma solução ótima por meio de métodos exatos (como a programação linear inteira) é, na maioria das vezes, inviável para hospitais de grande porte em tempo hábil [4, 6].

A natureza dinâmica do ambiente hospitalar exige que as decisões de alocação sejam tomadas em tempo real ou quase real, com soluções geradas em segundos, não em horas [6, 2]. Essa lacuna entre a complexidade do problema e a necessidade de respostas rápidas justifica o desenvolvimento de algoritmos heurísticos e meta-heurísticos, capazes de fornecer soluções de alta qualidade e próximas da ótima em um tempo viável [6, 7].

Diante do exposto, o objetivo central deste trabalho é o desenvolvimento de um algoritmo de otimização heurística para o problema de alocação de pacientes a leitos hospitalares, formulado como um modelo multiobjetivo de Programação Linear Inteira, sendo a função objetivo tratada como uma soma ponderada de penalidades.

Para contextualizar os resultados obtidos pela heurística, também foi implementada uma abordagem exata, na qual o mesmo modelo é resolvido pelo solver comercial Gurobi. Ambas as abordagens são comparadas quanto à qualidade da solução e ao esforço computacional em instâncias sintéticas de diferentes portes, de modo a identificar seu comportamento à medida que o tamanho do cenário simulado aumenta.

O modelo proposto, comum às abordagens heurística e exata, considera variáveis binárias que indicam em qual quarto cada paciente permanece em cada dia de sua internação, sendo as datas de admissão e de alta parte dos dados de entrada. Esse tem como objetivo minimizar simultaneamente as alocações em quartos de especialidade inadequada, as transferências durante a internação e a ocupação concomitante de quartos por pacientes de gêneros distintos, respeitando a capacidade de leitos de cada quarto.

Por fim, este trabalho se propõe a criar uma ferramenta de apoio à decisão que mantenha soluções de boa qualidade mesmo em instâncias de grande porte, típicas de grandes instituições hospitalares, nas quais os métodos exatos deixam de ser viáveis em tempo hábil.

Capítulo 2

Revisão Bibliográfica

2.1 Gestão de leitos: Problemas similares na literatura

O gerenciamento da capacidade hospitalar e, em particular, a alocação de leitos, pode ser mapeado para diversas famílias clássicas de problemas de otimização documentadas na literatura de Pesquisa Operacional. De acordo com a taxonomia de modelos de otimização detalhada por Talbi [7], o problema de alocação de leitos enquadra-se fortemente na categoria de Otimização Combinatória, compartilhando características fundamentais com os seguintes problemas clássicos:

- **Problemas de Atribuição (*Assignment Problems*):** Em sua essência, alocar pacientes a leitos é uma extensão do Problema de Atribuição Generalizada (GAP) ou do Problema de Atribuição Quadrática (QAP). No QAP, busca-se atribuir um conjunto de objetos a um conjunto de localizações minimizando custos de fluxo e distância [7]. No contexto hospitalar, os 'objetos' são os pacientes e as 'localizações' são os leitos/quartos, com o custo sendo definido por penalidades de alocação inadequada (ex.: falta de equipamentos) e a "distância" podendo representar o tempo de deslocamento da equipe médica [2].
- **Problemas de Escalonamento (*Scheduling Problems*):** O tempo de permanência do paciente (*Length of Stay* - LOS) insere a dimensão temporal no problema. Brandeau, Sainfort e Pierskalla [1] ressaltam que o planejamento de saúde frequentemente lida com o escalonamento de recursos escassos sob incerteza. Assim, a gestão de leitos assemelha-se ao *Job-Shop Scheduling* [7], onde os pacientes são as "tarefas" (com durações estocásticas) que precisam ser processadas em "máquinas" (leitos), respeitando datas de admissão e alta [6].
- **Problemas de Empacotamento (*Bin Packing*):** A alocação também pode ser vista sob a ótica do *Bin Packing*, onde o objetivo é empacotar itens de diferentes tamanhos no menor número de compartimentos. No hospital, os leitos ou alas hospitalares atuam como compartimentos de capacidade finita e a duração da internação ou a carga de trabalho de enfermagem exigida pelo paciente representa o tamanho do item [8, 7].

Essas similaridades teóricas são essenciais, pois permitem que a vasta gama de algoritmos já desenvolvidos para esses problemas clássicos na ciência da computação seja adaptada ao ambiente da saúde.

2.2 Classificação do problema

Na literatura internacional, o desafio de acomodar pacientes em unidades de internação é frequentemente referido pela sigla PBA (*Patient-Bed Allocation* - Alocação de Pacientes a Leitos) ou PAS (*Patient Admission Scheduling*).

O PBA não é simplesmente atribuir um leito vago a um paciente. Ele envolve uma série de critérios, restrições e objetivos conflitantes que afetam diretamente múltiplos *stakeholders* (pacientes, profissionais da saúde e administração), como:

- **Critérios Clínicos:** Alocar o paciente a um leito na unidade especializada correta (e.g., cardiologia, ortopedia, UTI), garantindo que os equipamentos médicos necessários estejam disponíveis [6, 2].
- **Restrições Operacionais:** Respeitar questões de gênero (leitos masculinos/femininos/mistos), idade (pediatria/geriatria), limites de capacidade do quarto e requisitos de isolamento (para pacientes com doenças infectocontagiosas ou baixa imunidade) [4, 2].
- **Preferências e Conforto:** Tentar alocar pacientes com perfis similares no mesmo quarto (adequação de idade), respeitar a preferência por quartos individuais ou enfermarias e minimizar transferências não planejadas de leito durante a internação, o que causa grande desconforto [6].
- **Eficiência dos Profissionais:** Agrupar pacientes de um mesmo médico ou especialidade para reduzir o tempo de deslocamento durante as visitas (rounds), e balancear a carga de trabalho da equipe de enfermagem, evitando que um único setor concentre todos os pacientes de alta complexidade [2].
- **Eficiência do Sistema:** Maximizar a taxa de ocupação (sem ultrapassar limites seguros), minimizar o tempo de espera ou atraso por um leito (evitando o *bed-block* no pronto-socorro), reduzir o tempo de higienização ou desocupação do leito e acomodar adequadamente pacientes eletivos e de emergência [1, 8].

2.2.1 Complexidade Computacional: O Desafio NP-Difícil

Devido à multiplicidade de variáveis, restrições e à natureza combinatória das alocações possíveis ao longo de um horizonte de tempo, o PBA é classificado na teoria da complexidade computacional como um problema **NP-Difícil** (*NP-Hard*) [4, 6, 2].

O que isso implica, na prática, é que o número de combinações possíveis para alocar P pacientes em L leitos ao longo de D dias cresce de forma exponencial. Segundo a teoria da complexidade detalhada por Talbi [7], para problemas da classe NP-Difícil, não existe nenhum algoritmo exato conhecido capaz de encontrar a solução ótima garantida em tempo polinomial. No contexto de um hospital de grande porte, isso significa que métodos exatos levariam horas, dias ou até semanas de processamento para definir o melhor arranjo de leitos. Contudo, o ambiente hospitalar é altamente dinâmico (com chegadas imprevisíveis de emergências e altas antecipadas/atrasadas), exigindo que as decisões de alocação sejam tomadas quase em tempo real, gerando respostas em segundos [6].

2.2.2 Modelagens na literatura

Diversos trabalhos na literatura tentaram enfrentar o PBA utilizando frameworks matemáticos formais, embora frequentemente esbarrem na barreira da complexidade computacional descrita acima.

As abordagens de **Programação Linear Inteira (PLI)** e **Programação Inteira Mista (MIP)** são as mais comuns na tentativa de modelar o problema de forma exata. Demeester et al. [6] desenvolveram um modelo matemático rigoroso para o PBA com o objetivo de minimizar as violações de restrições (como transferências e alocações em departamentos errados). Ao tentarem resolver o modelo através de um solver comercial padrão (ILOG CPLEX), não conseguiram encontrar sequer uma solução viável dentro de uma hora de cálculo e, mesmo após uma semana de processamento contínuo, a solução ótima não foi alcançada para instâncias de tamanho real.

Ceschia e Schaerf [4] também modelaram o PBA dinâmico sob incerteza através de Programação Inteira para estabelecer limites inferiores (*lower bounds*). Eles observaram que o modelo completo, mesmo com relaxamentos, exigia tempos proibitivos no CPLEX para instâncias médias e grandes, obrigando-os a fracionar as instâncias.

Para lidar com múltiplos objetivos conflitantes (custo, serviço, satisfação), abordagens de **Programação por Metas (Goal Programming)** e funções objetivo ponderadas têm sido amplamente adotadas. Beliën e Demeulemeester [8] formularam o problema de nivelamento da ocupação de leitos gerado pelo centro cirúrgico utilizando Programação Inteira Mista (MIP) e programação quadrática. Da mesma forma, Schäfer et al. [2] formularam uma função de utilidade total, combinando penalidades de médicos, enfermeiros e pacientes. Ressaltam que modelagens analíticas precisas devem ser integradas com lógicas de decisão rápidas para terem valor prático.

2.3 Abordagem proposta

Diante da complexidade combinatória intrínseca ao PBA (característica de problemas NP-Difíceis) e da ineficiência dos métodos exatos (como a Programação Inteira) para fornecer soluções em tempo viável, este trabalho adota a **otimização heurística** como abordagem central.

Conforme sugerido por Talbi [7], quando o tempo disponível para a tomada de decisão é crítico – como no gerenciamento de leitos, onde o fluxo de pacientes no pronto-socorro e no centro cirúrgico não pode parar – o uso de algoritmos aproximados (heurísticas e meta-heurísticas) é a via mais adequada. Embora essas técnicas não forneçam garantias matemáticas de que a solução encontrada é o ótimo global, elas exploram o espaço de busca de maneira inteligente e estruturada.

A abordagem heurística destaca-se por possuir um excelente potencial para encontrar soluções de alta qualidade (muito próximas do ótimo) em pouco tempo [4, 6]. Assim, a abordagem proposta visa atuar como um sistema de apoio à decisão tática e operacional, oferecendo o melhor balanço entre um alto rigor na satisfação das restrições clínicas/operacionais e o tempo de resposta exigido pela dinâmica do sistema de saúde.

2.4 Meta-heurísticas

Para lidar com a intratabilidade de problemas NP-Difíceis, como a alocação de leitos (PBA), as meta-heurísticas surgem como a principal alternativa aos métodos exatos. O termo "meta-heurística" define uma metodologia de nível superior que atua como uma estratégia guia para projetar heurísticas subjacentes aplicáveis a uma ampla variedade de problemas de otimização [7]. Embora não garantam a otimalidade global, elas fornecem soluções de alta qualidade ("aceitáveis") em um tempo de processamento razoável [7].

O design de qualquer meta-heurística envolve o balanceamento de dois critérios conflitantes: a *exploração* (diversificação), que garante que todas as regiões do espaço de busca sejam visitadas, e a *exploração* (intensificação), que refina as melhores soluções encontradas para gerar resultados ainda melhores [7]. De acordo com suas características de busca, as meta-heurísticas são tipicamente divididas em duas grandes famílias:

- **Meta-heurísticas baseadas em trajetória (S-metaheuristics):** Focam na transformação iterativa de uma única solução, sendo altamente orientadas à exploração (intensificação). A **Busca Tabu (Tabu Search - TS)** utiliza estruturas de memória (listas tabu) para registrar o histórico recente da busca, impedindo que o algoritmo visite soluções recém-avaliadas e fique preso em ótimos locais [7]. Demeester et al. [6] aplicaram um algoritmo de Busca Tabu híbrido para o problema de admissão de pacientes, demonstrando superioridade em relação a métodos exatos em tempo viável. O **Simulated Annealing (SA)**, inspirado no processo de recozimento de metais, utiliza uma função de probabilidade decrescente (temperatura) para aceitar soluções piores no início da busca, evitando ótimos locais e convergindo gradativamente para uma solução refinada [7]. Ceschia e Schaerf [4] utilizaram o SA para modelar e resolver o PAS dinâmico sob incerteza com excelentes resultados.
- **Meta-heurísticas baseadas em população (P-metaheuristics):** Lidam simultaneamente com um conjunto de soluções (população), sendo naturalmente orientadas à exploração (diversificação) do espaço de busca. Os **Algoritmos Genéticos (Genetic Algorithms - GA)** e outras abordagens evolutivas imitam a evolução biológica e a seleção natural, combinando soluções através de operadores de cruzamento (*crossover*) e mutação para gerar novas "descendências" [7, 9]. A **Inteligência de Enxame**, como a Otimização por Nuvem de Partículas (*Particle Swarm Optimization - PSO*) e Otimização por Colônia de Formigas (*Ant Colony Optimization - ACO*), baseia-se no comportamento descentralizado e auto-organizado de animais na natureza [7, 3, 9].

2.5 Combinação de algoritmos

A literatura demonstra que a aplicação de meta-heurísticas puras pode apresentar limitações ao lidar com espaços de busca altamente combinatórios e restritos. A hibridização, ou seja, a combinação de diferentes algoritmos, é uma estratégia adotada para suprir as fraquezas de um método com os pontos fortes de outro [7].

No contexto da saúde e do planejamento de capacidade, diversas abordagens híbridas obtiveram sucesso ao aliar qualidade e velocidade computacional:

- **Híbridos de P-metaheurísticas e S-metaheuristics:** Uma abordagem clássica (conhecida como Algoritmo Memético) incorpora um algoritmo de busca local dentro de um Algoritmo Genético. Enquanto o GA explora globalmente o espaço de soluções por meio da população, a busca local atua como um operador de "mutação inteligente", refinando cada indivíduo e acelerando a convergência para ótimos locais de alta qualidade [7].
- **Híbridos de Meta-heurísticas e Métodos Exatos:** Métodos baseados em Programação Matemática podem ser usados para resolver subproblemas específicos ou inicializar meta-heurísticas. Beliën e Demeulemeester [8] desenvolveram uma abordagem na qual o *Simulated Annealing* é inicializado com uma solução já muito boa, gerada previamente por um modelo de Programação Quadrática, garantindo melhor tempo computacional e qualidade do que o uso de ambos isoladamente.
- **Integração com *Machine Learning*:** Abordagens modernas combinam técnicas de mineração de dados e aprendizado de máquina para guiar a otimização. Schäfer et al. [2] integraram previsões de *Machine Learning* (sobre a chegada de pacientes de emergência) em uma *hiper-heurística*, que por sua vez gerencia heurísticas gulosas subjacentes, resultando em ganhos significativos no gerenciamento operacional dinâmico de leitos.

2.6 Etapa de decisão

Em problemas de saúde, formular e otimizar um modelo matemático é apenas parte do desafio; a fase crítica reside na etapa de tomada de decisão. A tomada de decisão é o processo iterativo onde os resultados algorítmicos são avaliados frente a cenários reais, levando à implementação de uma solução prática [7].

No gerenciamento de leitos, a etapa de decisão é de suma importância porque o problema é inerentemente **multiobjetivo** e sujeito a incertezas [2]. Uma otimização estrita baseada apenas em custos ou taxas de ocupação pode negligenciar fatores humanos vitais, como riscos à vida do paciente, carga de trabalho insalubre para enfermeiros ou ineficiência da rotina médica [2]. A etapa de decisão é o momento onde os múltiplos interesses (muitas vezes conflitantes) dos *stakeholders* são balanceados para garantir uma alocação que seja não apenas matematicamente ótima, mas clinicamente e operacionalmente viável.

2.6.1 Tomadores de decisão

Na literatura de otimização multiobjetivo, a interação entre o solucionador (o algoritmo) e o tomador de decisão (gestor hospitalar) pode ocorrer sob três abordagens principais [7]:

- **Abordagem *A Priori*:** O tomador de decisão estabelece suas preferências antes do início da otimização. Isso é frequentemente feito através da agregação das funções objetivo em uma única função de utilidade, onde cada critério recebe um peso. Essa abordagem é computacionalmente rápida (gera uma única solução e requer apenas uma execução da meta-heurística), mas exige que o gestor conheça antecipadamente o comportamento exato e a importância de cada variável [7, 2].

- **Abordagem A Posteriori:** O algoritmo executa a busca e devolve um conjunto de soluções de compromisso (a Fronteira de Pareto), representando diferentes cenários otimizados (por exemplo, uma solução excelente para médicos, mas razoável para pacientes; outra excelente para pacientes, mas ruim para enfermagem). Apenas no final o tomador de decisão escolhe a solução desejada. É muito útil quando o cenário não é claro, mas requer alto custo computacional [7].
- **Abordagem Iterativa:** Há uma cooperação progressiva entre o gestor e o algoritmo durante a busca. O algoritmo apresenta soluções parciais, o tomador de decisão refina suas preferências, e a busca é redirecionada. É altamente flexível, mas exige envolvimento constante do gestor [7].

2.7 Fechamento da revisão e definição da arquitetura

A gestão de leitos hospitalares foi confirmada ao longo desta revisão como um desafio estocástico, multiobjetivo e de complexidade NP-Difícil, tornando o uso de métodos exatos tradicionalmente inviáveis para decisões em tempo hábil [4, 6, 3].

Diante desse cenário, este trabalho adota os **Algoritmos Genéticos (GA)** como o motor central de otimização global. A escolha pelos GAs justifica-se fortemente por sua natureza baseada em população (*P-metaheuristic*), o que lhes confere a capacidade inerente de explorar o espaço de busca de forma ampla e de gerar, em uma única execução, uma diversidade de soluções candidatas [7]. Além disso, algoritmos genéticos possuem excelente maleabilidade para integrações multiobjetivo e permitem hibridizações com regras clínicas específicas.

Contudo, embora os GAs sejam métodos poderosos para a exploração global (*diversificação*), a literatura aponta que eles frequentemente apresentam limitações na fase de exploração (*intensificação*), ou seja, no ajuste fino das soluções para alcançar ótimos locais de alta qualidade [7]. Para suprir essa lacuna e lidar com o alto nível de restrições do PBA, este trabalho propõe uma arquitetura híbrida, frequentemente denominada **Algoritmo Memético** (classificada como um híbrido de baixo nível, ou *Low-Level Teamwork Hybrid*) [7].

Nesta abordagem, o GA é acoplado a um algoritmo de busca local baseada em múltiplas vizinhanças, sendo o *Variable Neighborhood Descent* (VND) o escolhido. A busca local não substitui o operador de mutação: a mutação do GA é mantida como fonte de diversidade e a busca local atua em seguida como um operador de intensificação aplicado a uma parcela da população a cada geração. Sob uma ótica Lamarckiana [7], o indivíduo refinado substitui o original na população, de modo que as melhorias obtidas pela busca local são herdadas pelas gerações seguintes por meio do cruzamento.

A escolha específica do VND como agente de busca local justifica-se pela topologia restrita do problema de alocação de leitos. Estudos em escalonamento de pacientes, como os conduzidos por Demeester et al. [6] e Ceschia e Schaerf [4], demonstraram que o uso de um único tipo de movimento (ex.: apenas trocar pacientes de quarto) é insuficiente para escapar de ótimos locais no ambiente hospitalar. O VND supera essa barreira ao explorar diferentes estruturas de vizinhança, alternando entre mudança de quarto, troca de pacientes e suas variantes parciais. Essa combinação funciona porque o GA garante que diferentes bacias de atração do espaço de busca sejam descobertas, enquanto o VND refina cada solução dentro do escopo local de sua bacia.

Quanto ao tomador de decisão, este trabalho adota uma postura a priori, por meio de um sistema de penalidades agregadas em uma única função de utilidade. Em um ambiente operacional hospitalar diário, a equipe de admissão precisa de respostas em tempo compatível com a sua rotina, o que inviabiliza análises a posteriori para cada novo paciente. Os pesos das restrições e dos interesses dos stakeholders são, portanto, pré-parametrizados [2], permitindo que o algoritmo recomende uma alocação de forma autônoma, sem intervenção do gestor durante a busca.

Com a base teórica acerca das características estruturais do problema estabelecida e a arquitetura híbrida (GA + VND) definida, o Capítulo 3 detalha a metodologia, apresentando o modelo matemático adotado e as estratégias de implementação das abordagens exata e heurística.

Capítulo 3

Metodologia

Este capítulo apresenta a metodologia utilizada para modelar e resolver o problema de alocação de leitos hospitalares. Primeiramente, são descritos o problema e as instâncias de teste; em seguida, o modelo matemático, resolvido pela abordagem exata; depois, a representação da solução e os operadores do Algoritmo Memético, base da abordagem heurística, e a sua implementação; por fim, as métricas de avaliação e o protocolo experimental.

3.1 Descrição do problema

O Problema de Alocação de Leitos (PBA) abordado neste trabalho concentra-se no nível operacional e tático da gestão hospitalar. O objetivo diário da equipe de admissão é alocar um conjunto de pacientes, com diferentes necessidades clínicas e tempos de permanência, a um conjunto finito de quartos e leitos disponíveis nas diversas alas do hospital [6]. A alocação deve garantir a viabilidade clínica e, simultaneamente, maximizar o conforto do paciente e a eficiência dos recursos operacionais [2].

3.1.1 Parâmetros, variáveis e discretização do tempo

Para reduzir a complexidade computacional e refletir a rotina de censos hospitalares, o tempo é discretizado e modelado em dias consecutivos [4, 6]. O dia atua como a menor unidade indivisível de tempo do sistema. Dessa forma, as internações, altas e transferências ocorrem estritamente na transição de um dia para o outro [6].

O sistema recebe como entrada uma lista de pacientes que possuem uma data de admissão e uma data de alta prevista, resultando em um Tempo de Permanência (LOS - *Length of Stay*) ininterrupto [4]. Durante todo o LOS estipulado, o paciente deve obrigatoriamente permanecer alocado nas dependências do hospital, não sendo consideradas transferências para unidades externas.

O hospital é composto por quartos, cada um associado a uma especialidade clínica e com uma capacidade máxima definida (número de leitos). Como não há diferenciação de equipamentos vitais ou infraestrutura entre leitos de um mesmo quarto, os leitos tornam-se indistinguíveis. Conforme proposto por Ceschia e Schaerf [4], o problema pode ser simplificado passando de uma alocação baseada em leitos individuais para uma alocação direta em quartos, o que reduz substancialmente o espaço de busca. Assim, as variáveis de decisão centrais definem em qual quarto o paciente passará cada dia do seu período de internação.

3.1.2 Restrições

O problema está sujeito a dois conjuntos de regras [4, 6]:

- **Restrições Rígidas (*Hard Constraints*):** Devem ser obrigatoriamente satisfeitas para que a solução seja viável:
 - *Garantia de Internação Contínua:* todo paciente admitido deve ocupar exatamente um quarto em cada dia de seu tempo de permanência.
 - *Capacidade do Quarto:* o número de pacientes alocados simultaneamente em um quarto não pode exceder a capacidade máxima (número de leitos) daquele quarto. Na abordagem heurística, essa restrição é relaxada e penalizada (ver Seção 3.3).
- **Restrições Flexíveis (*Soft Constraints*):** Representam preferências que, se violadas, geram penalidades na função objetivo, sem inviabilizar a solução:
 - *Especialidade:* alocar o paciente em um quarto da especialidade clínica adequada à sua patologia.
 - *Transferências:* evitar que o paciente mude de quarto durante a internação.
 - *Política de gênero:* evitar que pacientes de gêneros distintos ocupem, concomitantemente, o mesmo quarto.

Outras regras citadas na literatura, como agrupamento por faixa etária, requisitos de isolamento e a preferência do paciente por quarto individual ou enfermaria, não foram consideradas neste trabalho.

3.1.3 Pesos e abordagem multiobjetivo

A natureza multiobjetivo do PBA exige o balanceamento de interesses conflitantes (ex.: minimizar transferências vs. agrupar pacientes da mesma especialidade). Conforme discutido na fundamentação teórica, este trabalho adota uma abordagem *a priori* [7]. Todos os objetivos e violações de restrições flexíveis são agregados em uma única função de utilidade através do Método das Somas Ponderadas [2, 6]. Dessa forma, pesos específicos (W) são atribuídos a cada violação, refletindo as prioridades da administração hospitalar. Quanto maior o peso de um critério (como a penalidade por transferências), maior será o esforço do algoritmo para evitá-lo.

3.1.4 Dados e Instâncias de Teste

Para testar e validar o algoritmo proposto, a literatura aponta a dificuldade de obtenção de dados hospitalares reais e abertos, devido a questões de sigilo médico [6].

Dessa forma, as instâncias de teste foram geradas sinteticamente por um gerador próprio, descrito na Subseção 3.1.5. Os parâmetros do gerador, como a ocupação-alvo de 90% e o tempo médio de permanência de quatro dias, foram escolhidos para produzir cenários de alta ocupação, nos quais as decisões de alocação têm maior impacto.

Além disso, com o objetivo de isolar e avaliar estritamente a performance computacional e a qualidade da otimização das heurísticas aplicadas, este trabalho opta pela utilização de

dados fixos e determinísticos, em que cada instância é gerada a partir de uma semente fixa, o que torna os experimentos reproduzíveis. Conforme a metodologia validada por Schäfer et al. [2], a eliminação dos fatores estocásticos (como variações inesperadas no tempo de permanência ou cancelamentos repentinos) durante a fase de validação algorítmica permite monitorar com clareza a eficácia da meta-heurística. Assim, as taxas de chegada, as capacidades dos quartos e os tempos de internação são predefinidos e fornecidos ao modelo como parâmetros exatos.

3.1.5 Gerador de instâncias

As instâncias de teste são produzidas por um gerador sintético desenvolvido neste trabalho, em quatro portes: pequena, média, grande e muito grande, com 10, 40, 160 e 640 quartos, respectivamente. Todos os quartos têm capacidade de três leitos, e cada instância contém dez pacientes por quarto, totalizando de 100 a 6400 pacientes. Para cada porte são gerados dez conjuntos independentes, o que permite avaliar a variação dos resultados devida ao próprio sorteio dos dados, e não apenas à aleatoriedade da heurística.

Cada conjunto é gerado a partir de uma semente própria, calculada como $100b + k$, em que b é a semente base do porte (42, 43, 44 e 45, respectivamente) e $k \in \{1, \dots, 10\}$ é o número do conjunto. Dessa forma, toda execução do gerador reproduz exatamente as mesmas instâncias, e nenhum par de conjuntos compartilha a mesma semente.

A especialidade de cada quarto e a especialidade requerida por cada paciente são sorteadas com a mesma distribuição: 40% Clínica Geral e 20% para cada uma das especialidades de Cirurgia, Cardiologia e Ortopedia. O gênero é sorteado com probabilidade igual entre feminino e masculino. O tempo de permanência é sorteado a partir de uma distribuição normal com média de quatro dias e desvio padrão de um dia, arredondado para o inteiro mais próximo e limitado inferiormente a um dia.

O dia de admissão de cada paciente não é sorteado, mas definido por uma estratégia de empacotamento, na qual os pacientes são ordenados do maior para o menor tempo de permanência, com desempate aleatório, e cada um é admitido no primeiro dia a partir do qual há leito livre no hospital durante toda a sua internação. O horizonte inicial é dimensionado para uma ocupação-alvo de 90% da capacidade total:

$$H = \left\lceil \frac{|P| \cdot \overline{LOS}}{0,9 \cdot \sum_{r \in R} c_r} \right\rceil + 2 \quad (3.1)$$

em que $\overline{LOS}$ é o tempo médio de permanência sorteado e, caso algum paciente não caiba no horizonte, ele é estendido. Definir a admissão por empacotamento, e não por sorteio, evita que a capacidade fique fragmentada ao longo do horizonte, o que reduziria a ocupação efetiva abaixo do alvo. Além disso, como o número de pacientes internados em cada dia nunca excede o total de leitos do hospital, toda instância gerada admite ao menos uma solução viável para o modelo da Seção 3.2.

3.2 Modelagem Matemática

A formulação a seguir define o Problema de Alocação de Leitos (PBA) em sua totalidade matemática como um modelo de Programação Linear Inteira (PLI). Este modelo estabelece

o rigor analítico do problema e é resolvido diretamente pelo solver comercial Gurobi na abordagem exata.

A formulação adota a abordagem de linearização proposta por Ceschia e Schaerf [5] para rastrear as características dinâmicas de convivência nos quartos (como a política de isolamento de gêneros), introduzindo variáveis auxiliares que definem o estado de ocupação de cada quarto a cada dia.

3.2.1 Conjuntos e Parâmetros

O modelo é fundamentado nos seguintes conjuntos, índices e parâmetros:

- P : Conjunto de todos os pacientes. Este conjunto é subdividido em pacientes do sexo feminino (P_F) e masculino (P_M), de modo que $P = P_F \cup P_M$.
- R : Conjunto de quartos disponíveis no hospital.
- D : Conjunto de dias do horizonte de planejamento (dia 1 ao último dia de alta entre todos os pacientes).
- $D_p = \{d_p^{\text{adm}}, \dots, d_p^{\text{alta}}\} \subseteq D$: Subconjunto de dias contínuos em que o paciente p está internado, sendo d_p^{adm} o dia de admissão e d_p^{alta} o último dia de internação.
- c_r : Capacidade máxima (número de leitos) do quarto r .
- $C_{p,r}$: Custo de alocação do paciente p ao quarto r . Este parâmetro penaliza alocações indesejadas, devido a inadequação da especialidade clínica do quarto ao diagnóstico do paciente [5].

$$C_{p,r} = \begin{cases} W_{\text{spec}}, & \text{se } e_p \neq e_r \\ 0, & \text{caso contrário} \end{cases} \quad (3.2)$$

em que e_p é a especialidade clínica requerida pelo paciente p e e_r é a especialidade do quarto r .

- W_{spec} : Peso de penalidade por dia em que o paciente permanece em um quarto de especialidade inadequada.
- W_{transf} : Peso de penalidade atribuído a cada transferência de quarto sofrida pelo paciente durante a internação.
- W_{gen} : Peso de penalidade por violação da política de gênero (alocar pacientes de sexos distintos no mesmo quarto no mesmo dia).

Os pesos adotados neste trabalho refletem a prioridade atribuída a cada um dos critérios considerados na função objetivo, sendo a adequação da especialidade o critério mais relevante, seguido da política de gênero e, por fim, do desconforto causado por uma transferência. Dessa forma, são adotados os valores apresentados na Tabela 3.1.

Tabela 3.1: Pesos adotados para os critérios da função objetivo

Pesos	Valor
W_{spec}	100
W_{gen}	75
W_{transf}	50

3.2.2 Variáveis de Decisão

O modelo baseia-se em variáveis primárias para a alocação e variáveis auxiliares para a linearização dos custos [5].

Variáveis Principais (Alocação e Transferência):

$$x_{p,r,d} \in \{0, 1\} \quad \text{Vale 1 se o paciente } p \text{ está alocado no quarto } r \text{ no dia } d. \quad (3.3)$$

$$t_{p,r,d} \in \{0, 1\} \quad \text{Vale 1 se o paciente } p \text{ é transferido do quarto } r \text{ do dia } d \text{ para o dia } d + 1. \quad (3.4)$$

Variáveis de Linearização (Status do Quarto): Para rastrear a mistura de gêneros de forma linear, definem-se as variáveis de status do quarto r no dia d :

$$f_{r,d} \in \{0, 1\} \quad \text{Vale 1 se há pelo menos uma paciente mulher no quarto } r \text{ no dia } d. \quad (3.5)$$

$$m_{r,d} \in \{0, 1\} \quad \text{Vale 1 se há pelo menos um paciente homem no quarto } r \text{ no dia } d. \quad (3.6)$$

$$b_{r,d} \in \{0, 1\} \quad \text{Vale 1 se o quarto } r \text{ no dia } d \text{ possui pacientes de ambos os gêneros.} \quad (3.7)$$

3.2.3 Função Objetivo

A função objetivo Z visa minimizar a soma total das penalidades. Ela é composta pelos custos clínicos da alocação ($C_{p,r}$), pelas penalidades aplicadas a cada transferência de quarto realizada (W_{transf}) e pelas penalidades geradas pelo uso de quartos mistos (W_{gen}):

$$\min Z = \sum_{p \in P} \sum_{r \in R} \sum_{d \in D_p} (C_{p,r} \cdot x_{p,r,d}) + \sum_{p \in P} \sum_{r \in R} \sum_{d \in D_p} (W_{\text{transf}} \cdot t_{p,r,d}) + \sum_{r \in R} \sum_{d \in D} (W_{\text{gen}} \cdot b_{r,d}) \quad (3.8)$$

3.2.4 Restrições

O modelo está sujeito às seguintes restrições:

1. Atribuição Única e Contínua: Cada paciente p deve estar alocado a exatamente um quarto durante todos os dias contínuos da sua internação (D_p).

$$\sum_{r \in R} x_{p,r,d} = 1, \quad \forall p \in P, \forall d \in D_p \quad (3.9)$$

2. Limite de Capacidade do Quarto: O número total de pacientes acomodados simultaneamente em um quarto r no dia d não pode exceder sua capacidade máxima c_r .

$$\sum_{p \in P} x_{p,r,d} \leq c_r, \quad \forall r \in R, \forall d \in D \quad (3.10)$$

3. Rastreamento de Transferências: A variável de transferência deve ser ativada caso o paciente ocupe o quarto r no dia d , mas não esteja mais alocado nele no dia subsequente $d + 1$ da sua internação [5].

$$t_{p,r,d} \geq x_{p,r,d} - x_{p,r,d+1}, \quad \forall p \in P, \forall r \in R, \forall d \in D_p \setminus \{d_p^{\text{alta}}\} \quad (3.11)$$

4. Identificação de Presença Feminina: Se uma paciente do conjunto feminino (P_F) for alocada ao quarto r no dia d , a flag indicadora de presença feminina desse quarto deve ser ativada.

$$f_{r,d} \geq x_{p,r,d}, \quad \forall p \in P_F, \forall r \in R, \forall d \in D_p \quad (3.12)$$

5. Identificação de Presença Masculina: Semelhantemente à restrição anterior, a flag deve ser ativada para pacientes do conjunto masculino (P_M).

$$m_{r,d} \geq x_{p,r,d}, \quad \forall p \in P_M, \forall r \in R, \forall d \in D_p \quad (3.13)$$

6. Detecção de Mistura de Gêneros (Quarto Misto): Esta restrição força a variável $b_{r,d}$ a assumir o valor 1 caso o quarto possua representantes de ambos os gêneros no mesmo dia, acionando a penalidade W_{gen} na função objetivo [5].

$$b_{r,d} \geq m_{r,d} + f_{r,d} - 1, \quad \forall r \in R, \forall d \in D \quad (3.14)$$

3.2.5 Domínio das Variáveis

Conforme definido anteriormente, o domínio de todas as variáveis restringe-se a valores binários:

$$x_{p,r,d}, t_{p,r,d}, f_{r,d}, m_{r,d}, b_{r,d} \in \{0, 1\} \quad (3.15)$$

3.3 Representação da solução

Um dos passos mais críticos no design de qualquer meta-heurística, especialmente em Algoritmos Genéticos (GA), é a escolha de uma representação de solução (cromossomo) que seja ao mesmo tempo eficiente em termos de memória e fácil de ser manipulada pelos operadores de busca [7].

Para o Problema de Alocação de Leitos (PBA), a representação computacional de um indivíduo na população precisa espelhar a variável de decisão $x_{p,r,d}$. Em vez de utilizar uma matriz tridimensional esparsa, que consumiria processamento excessivo, a solução é representada como um *array* (ou dicionário de dados), onde cada índice representa um paciente $p \in P$. O valor (alelo) contido nesse índice é um subvetor que lista o quarto $r \in R$ ocupado pelo paciente em cada dia contínuo $d \in D_p$ do seu tempo de permanência (LOS) [6, 4].

Solução inicial:

Em algoritmos genéticos e de busca local, a qualidade da população inicial impacta diretamente a velocidade de convergência. Neste trabalho, a população inicial combina dois construtores com vieses distintos. A maior parte dos indivíduos (80%) é gerada de forma semigulosa e aleatória [4], onde, para cada paciente, sorteia-se um quarto da especialidade que

ele requer, mantendo-o no mesmo quarto do primeiro ao último dia de internação ($t_{p,r,d} = 0$), sem considerar a capacidade.

Os 20% restantes são construídos de forma gulosa e ciente da capacidade: os pacientes são processados em ordem aleatória, cada um é alocado a um quarto de sua especialidade com leito livre em todos os dias da internação, não havendo, a qualquer quarto com leito livre e, só em último caso, a um quarto sem vaga. Essa proporção foi definida experimentalmente, como um equilíbrio entre manter a diversidade da população e garantir a presença de indivíduos viáveis nas instâncias maiores.

Na abordagem heurística, diferentemente da abordagem exata, a capacidade dos quartos é tratada como uma restrição flexível. Rejeitar toda solução que excedesse a capacidade criaria descontinuidades no espaço de busca e restringiria excessivamente os operadores. Em vez disso, o excesso de ocupação é contabilizado pela violação de capacidade V_{cap} , que soma, para cada quarto e cada dia, quantos pacientes excedem o número de leitos disponíveis:

$$V_{cap} = \sum_{r \in R} \sum_{d \in D} \max \left(0, \sum_{p \in P} x_{p,r,d} - c_r \right) \quad (3.16)$$

em que $\sum_{p \in P} x_{p,r,d}$ é o número de pacientes alocados no quarto r no dia d , e o termo $\max(0, \cdot)$ considera apenas a parcela que ultrapassa a capacidade c_r , sendo nulo quando o quarto está dentro do limite. Cada unidade de V_{cap} corresponde, portanto, a um paciente-dia alocado além do número de leitos do quarto, sendo uma solução viável se, e somente se, $V_{cap} = 0$.

Os indivíduos são então comparados pelo *fitness* F , que acrescenta à função objetivo a violação de capacidade ponderada:

$$F = Z + W_{cap} \cdot V_{cap} \quad (3.17)$$

em que Z é o valor da função objetivo (Equação 3.8) e W_{cap} é o peso de penalidade atribuído a cada paciente-dia alocado além da capacidade.

Como W_{cap} não faz parte da função objetivo (Z), o valor comparado ao Gurobi é sempre Z e a violação de capacidade V_{cap} é reportada separadamente. Adotou-se $W_{cap} = 1900$, valor obtido por uma varredura experimental (Subseção 3.6.1). Como a melhor solução pelo fitness pode violar a capacidade, o algoritmo também registra a melhor solução viável encontrada em qualquer geração.

Estruturas de vizinhança:

Na arquitetura do algoritmo memético adotada, a busca local (VND) é aplicada a cada geração, após o cruzamento e a mutação. Para que o VND consiga escapar de ótimos locais em um ambiente altamente restrito como o de um hospital, ele deve alternar sistematicamente entre diferentes topologias de vizinhança [7].

Baseando-se nas abordagens consolidadas por Ceschia e Schaerf [4] e Demeester [6], para o agendamento dinâmico de pacientes em quartos, as seguintes estruturas de vizinhança formam o núcleo do otimizador local:

- **N_1 : Mudança de Quarto (*Change Room - CR*):** Este movimento consiste em transferir virtualmente um paciente p de seu quarto atual para um novo quarto r' durante **toda** a sua estadia. Trata-se de uma exploração primária do espaço de busca (S_0), mantendo

o número de transferências intrahospitalares igual a zero, mas buscando quartos que reduzam o custo estático $C_{p,r}$ [4, 6].

- **N_2 : Troca entre Pacientes (*Swap Patients - SP*):** Consiste em permutar os quartos atribuídos a dois pacientes distintos p_1 e p_2 , cujos dias de internação possuam alguma sobreposição. Esse movimento é vital pois, frequentemente, a mudança de um paciente isolado (N_1) é inviabilizada por limites de capacidade. A troca simultânea permite que o algoritmo contorne bloqueios de leitos e acomode melhor os pacientes sem inflar a ocupação máxima do quarto [4, 6].
- **N_3 : Mudança Parcial de Quarto (*Partial Change Room - PCR*):** As vizinhanças anteriores exploram apenas soluções sem transferências. No entanto, quartos ideais podem estar disponíveis apenas em parte do período de internação. O movimento PCR atua no espaço de soluções expandido (S_1) ao selecionar um dia d no meio da internação de um paciente p e alterar seu quarto apenas daquele dia em diante. Embora esse movimento ative a variável de transferência $t_{p,r,d}$ e adicione a penalidade de transferência (W_{transf}) à Função Objetivo, ele confere a flexibilidade necessária para resolver gargalos de capacidade em dias de pico [4].
- **N_4 : Troca Parcial entre Pacientes (*Partial Swap Patients – PSP*):** generalização de N_2 proposta por Ceschia e Schaerf [5]. Em vez de permutar as internações inteiras de dois pacientes, troca os quartos apenas nos dias em que as internações se sobrepõem, preservando o restante da sequência de cada um. Com isso, resolve conflitos restritos a parte da internação sem desfazer transferências já construídas por N_3 .

A busca local (VND) percorre os pacientes um a um e, para cada paciente, tenta as vizinhanças na ordem $N_1 \rightarrow N_2 \rightarrow N_3 \rightarrow N_4$, aceitando o primeiro movimento que melhore o fitness. Para reduzir o custo desse processo, adotam-se três mecanismos:

1. Don't-look bits, em que um paciente sem movimento de melhora fica inativo e só volta a ser examinado se um movimento alterar sua alocação.
2. Amostragem de candidatos, em que cada movimento avalia no máximo 10 quartos, sorteados entre 30 e priorizados pelos que têm leito livre nos dias em questão.
3. Limite de 250 avaliações por paciente em cada chamada, que impede que o refino de um único indivíduo consuma uma geração inteira nas instâncias grandes.

Operadores do algoritmo genético:

A população tem tamanho fixo de 30 indivíduos em todos os portes de instância. A cada geração, o algoritmo executa os passos a seguir, até que o teto de tempo seja atingido:

1. **Elitismo:** os 10% melhores indivíduos da geração anterior são copiados para a nova geração.
2. **Reprodução:** o restante da população é preenchido por filhos gerados a partir de dois pais escolhidos por torneio de tamanho 3, aos quais se aplicam cruzamento e mutação.

3. **Imigrantes:** 10% dos filhos não elite são substituídos por indivíduos novos, gerados pelo construtor aleatório da solução inicial.
4. **Busca local:** o VND é aplicado a todos os elites e a 30% dos demais indivíduos, sorteados. Os elites são incluídos porque, como não passam por cruzamento nem mutação, permaneceriam inalterados enquanto mantivessem sua posição na população, escolha validada experimentalmente (Subseção 3.6.1).
5. **Atualização:** a população é ordenada pelo fitness, e são registradas a melhor solução e a melhor solução viável encontradas até então.

Os operadores de cruzamento e de mutação usados no passo de reprodução foram adaptados à estrutura do problema e são descritos a seguir.

- **Cruzamento por blocos de quarto:** Com probabilidade de 0,9, o filho é gerado por cruzamento; caso contrário, é uma cópia de um dos pais. No cruzamento, um dos pais é sorteado como primário, e os pacientes são agrupados de acordo com o quarto que ocupam no primeiro dia de internação nesse pai. Cada grupo é herdado em bloco do pai primário com probabilidade de 0,5, e os pacientes dos demais grupos recebem a sequência de quartos que tinham no outro pai. Herdar o grupo inteiro preserva as combinações de colegas de quarto já ajustadas, especialmente quanto ao gênero, que um cruzamento paciente a paciente tenderia a desfazer.
- **Mutação:** Cada paciente do filho é mutado com probabilidade de 5%, por um movimento sorteado entre três tipos análogos às vizinhanças do VND: mudança do quarto em toda a internação (N1), mudança a partir de um dia sorteado (N3) ou troca de quartos com outro paciente nos dias em que ambos estão internados (N4). Ao contrário da busca local, a mutação não verifica se o movimento melhora a solução. Essa intensidade é ajustada conforme o progresso da busca: quando a melhor solução não melhora por 5 gerações consecutivas, a mutação passa a alterar exatamente 35% dos pacientes de cada filho, voltando aos 5% assim que uma nova melhor solução é encontrada. Como o algoritmo só termina pelo teto de tempo, esse reforço pode ser acionado várias vezes em uma mesma execução.

Em conjunto, esses mecanismos equilibram as duas forças da busca: o elitismo e o VND intensificam a busca em torno das melhores soluções, enquanto a mutação, seu reforço por estagnação e os imigrantes mantêm a diversidade da população. Os parâmetros adotados estão resumidos na Tabela 3.2.

Tabela 3.2: Parâmetros do Algoritmo Memético

Parâmetro	Valor
Tamanho da população	30
População inicial gulosa / aleatória	20% / 80%
Tamanho do torneio	3 indivíduos
Taxa de cruzamento	0,9
Probabilidade de herança de um bloco	0,5
Taxa de mutação	5% dos pacientes
Mutação reforçada	35% dos pacientes, após 5 gerações sem melhora
Elites	10% da população
Imigrantes	10% da população
Busca local	Elites + 30% dos demais
Candidatos por movimento do VND	10 quartos, sorteados entre 30
Limite de avaliações do VND	250 por paciente
W_{cap}	1900

3.4 Implementação

Avaliação Procedural e Simplificação do Modelo

Embora a Seção 3.2 apresente a formulação matemática exata e completa do problema por meio de Programação Linear Inteira (PLI), necessária para a execução com solvers comerciais como o Gurobi, a implementação computacional da meta-heurística pode adotar uma abordagem mais enxuta para a avaliação da função objetivo [6].

Em modelos estritamente lineares, a detecção de penalidades dinâmicas, como a violação da política de gênero no mesmo quarto, exige a criação de múltiplas variáveis de decisão auxiliares (e.g., $f_{r,d}$, $m_{r,d}$ e $b_{r,d}$) e restrições de amarração para cada quarto e cada dia do horizonte de planejamento [5]. Para um hospital de grande porte, isso resulta em uma explosão combinatória do número de variáveis, tornando o modelo intratável em tempo hábil [6].

Na arquitetura do Algoritmo Memético implementada neste trabalho (em Python), contorna-se essa limitação através da **avaliação procedural**. Como a meta-heurística explora o espaço de busca gerando arranjos completos (ou vizinhanças) a cada iteração, o algoritmo possui conhecimento total e imediato sobre o estado de ocupação de qualquer quarto r no dia d .

Dessa forma, a heurística dispensa a declaração das variáveis auxiliares de linearização. O custo é mantido de forma incremental: a solução guarda, para cada quarto e dia, o conjunto de ocupantes e a contagem por gênero, e cada movimento atualiza apenas os quartos e dias que foram alterados. Assim, avaliar um movimento custa proporcionalmente ao tempo de permanência do paciente movido, e não ao tamanho da instância.

Essa diferença entre a abordagem exata, que representa o modelo por restrições algébricas, e a abordagem heurística, que o avalia por meio de estruturas de dados, permite ao algoritmo memético avaliar entre cerca de 50 e 75 mil soluções candidatas por segundo. A taxa diminui apenas moderadamente com o porte, de aproximadamente 73 mil na instância pequena para 51 mil na muito grande, embora esta tenha 64 vezes mais pacientes.

Ferramentas e Ambiente de Desenvolvimento

O algoritmo heurístico proposto, bem como a modelagem de dados e rotinas de avaliação, foram desenvolvidos em Python 3.13, juntamente com as bibliotecas NumPy, Pandas e Matplotlib, com o solver exato executado com o Gurobi 13.0.1.

Para o controle de versionamento e armazenamento do código-fonte, utiliza-se a ferramenta Git, com o projeto hospedado publicamente na plataforma **GitHub**. O repositório oficial do trabalho pode ser acessado através do [link](#).

O ambiente de desenvolvimento foi estruturado seguindo boas práticas de engenharia de software para garantir a reprodutibilidade dos experimentos. A execução do projeto é isolada através de um ambiente virtual (`venv`), com todas as dependências bibliotecárias e suas respectivas versões documentadas no arquivo `requirements.txt`. A arquitetura do código está modularizada no diretório `src`, onde o script `main.py` atua como o orquestrador principal da otimização, interligando a fonte de dados, a avaliação da função objetivo e os operadores do algoritmo memético (GA + VND).

3.5 Métricas para avaliação de desempenho

A avaliação de desempenho de algoritmos heurísticos exige um planejamento experimental rigoroso para comprovar sua eficácia e eficiência frente ao problema proposto. Conforme definido por Talbi [7], para a análise de algoritmos de otimização, o desempenho do Algoritmo Memético desenvolvido é medido sob três perspectivas principais: qualidade da solução, eficiência computacional e robustez estatística.

- **Eficácia (Qualidade da Solução):** A qualidade das alocações geradas é avaliada primariamente pelo valor final alcançado na função objetivo Z (Equação 3.8). Como o problema de alocação de leitos é de minimização, valores menores indicam um melhor balanceamento das restrições flexíveis. Adicionalmente, para cada solução são registrados o valor de cada parcela de Z e métricas operacionais explícitas, como o número de pacientes-dia em quarto de especialidade inadequada, o número de transferências, o número de quartos-dia mistos, a violação de capacidade (V_{cap}), a taxa de ocupação média e por especialidade e o pico de ocupação diária do hospital.

Como a abordagem heurística trata a capacidade como restrição flexível, nem toda execução termina com uma solução viável. Nesses casos, é preciso medir o quão distante a solução está da viabilidade, e o valor absoluto de V_{cap} (Equação 3.16) não basta para isso, pois tende a crescer com o porte da instância. Por esse motivo, V_{cap} é comparada à carga de internação N_{pd} , isto é, ao número total de pacientes-dia da instância:

$$N_{pd} = \sum_{p \in P} |D_p| \quad (3.18)$$

em que $|D_p|$ é o número de dias de internação do paciente p . Como cada unidade de V_{cap} corresponde a um paciente-dia, a violação nunca excede a carga de internação ($V_{cap} \leq N_{pd}$), que funciona, assim, como seu limite superior. A razão entre as duas grandezas define a taxa de violação de capacidade ρ_{cap} :

$$\rho_{cap} = \frac{V_{cap}}{N_{pd}} \times 100\% \quad (3.19)$$

que expressa a porcentagem da demanda de internação que ficou sem leito. Assim, $\rho_{cap} = 0\%$ indica uma solução viável, e valores próximos de zero indicam soluções próximas da viabilidade.

Para atestar a eficácia do algoritmo memético, as soluções heurísticas são comparadas às obtidas pelo método exato, com o solver Gurobi, executado sobre o modelo completo da Seção 3.2 com o mesmo teto de tempo concedido a cada execução da heurística. Ao final do tempo, o Gurobi fornece a melhor solução inteira encontrada (limite superior), o melhor limite inferior do branch-and-bound e o gap de otimalidade entre eles. Quando o gap é nulo, a solução é ótima comprovada; caso contrário, o limite inferior indica o quanto, no máximo, qualquer solução poderia melhorar.

Optou-se pelo limite do branch-and-bound em vez da relaxação linear isolada [5] porque esta se mostrou impraticável nas instâncias maiores. Na instância grande (1600 pacientes), a relaxação não foi resolvida em 300 segundos e, na muito grande (6400 pacientes), apenas a construção do modelo, com cerca de 28,6 milhões de variáveis, consumiu quase 300 segundos.

Nas instâncias maiores, porém, o Gurobi pode não avançar além da relaxação inicial dentro do teto de tempo. Nesses casos, o limite inferior permanece trivial (igual a zero), e a comparação se restringe à melhor solução inteira encontrada, o que é coerente com a limitação dos métodos exatos discutida na Subseção 2.2.1 e na Seção 3.4.

- **Eficiência Computacional:** Ambos os métodos recebem o mesmo teto de tempo por execução, que cresce com o tamanho da instância (Seção 3.6). A heurística não possui critério de parada por estagnação e sempre utiliza o teto completo, de modo que o tempo de execução, por si só, não diferencia os métodos. Por isso, o esforço computacional é medido pelo número de avaliações da função objetivo dentro do teto. No Gurobi, esse número corresponde às iterações do simplex; na heurística, à soma das avaliações feitas pelo GA e pela busca local. A velocidade de convergência é acompanhada pela curva do melhor fitness a cada geração, pela geração em que a melhor solução viável foi encontrada e pela comparação com o custo da população inicial. Os tetos adotados, de 30 a 900 segundos, foram definidos para comparar os métodos em condições iguais e não representam o tempo de resposta esperado em operação.
- **Análise Estatística e Robustez:** Devido à natureza estocástica dos Algoritmos Genéticos e à aleatoriedade na exploração de vizinhanças do VND, uma única execução não é suficiente para atestar a qualidade do método [7]. Portanto, cada conjunto de dados é resolvido por 10 execuções independentes da heurística (5 na instância muito grande, por limitação de tempo computacional), cada uma com sua própria sequência aleatória. Para cada conjunto, reportam-se o melhor valor, a média e o desvio padrão dessas execuções, sendo a dispersão visualizada em gráficos do tipo boxplot. Além disso, cada porte de instância é composto por 10 conjuntos gerados independentemente, o que permite separar duas fontes de variação: a variação entre execuções, devida à aleatoriedade da heurística, e a variação entre conjuntos, devida ao sorteio da instância.

3.6 Protocolo experimental

Para cada porte de instância, cada um dos dez conjuntos é resolvido uma vez pela abordagem exata e várias vezes pela abordagem heurística, com o mesmo teto de tempo por execução para as duas. A Tabela 3.3 resume a configuração adotada.

Tabela 3.3: Configuração experimental por porte de instância

Porte	Quartos	Pacientes	Teto por execução	Execuções da heurística	Conjuntos
Pequena	10	100	30 s	10	10
Média	40	400	90 s	10	10
Grande	160	1600	360 s	10	10
Muito grande	640	6400	900 s	5	10

O teto de tempo cresce com o porte da instância. Entre as instâncias média e grande, ele foi aumentado na mesma proporção que o número de pacientes, o que manteve aproximadamente 15 gerações por execução da heurística nos dois portes. Pelo mesmo critério, a instância muito grande deveria receber 1440 segundos, contudo, em razão do tempo computacional total, foram adotados 900 segundos e 5 execuções por conjunto, o que reduz o número de gerações disponíveis nesse porte (8 por execução, contra cerca de 15 nas instâncias média e grande) e deve ser considerado na análise dos resultados.

As instâncias têm semente fixa (Subseção 3.1.5), mas cada execução da heurística utiliza uma sequência aleatória própria, de modo que as execuções de um mesmo conjunto constituem amostras independentes. Na abordagem exata, o teto de tempo se aplica à otimização, mas, a fim de possibilitar a geração de algum resultado, o tempo de construção do modelo no Gurobi não é contabilizado. Na instância muito grande, por exemplo, essa construção leva quase cinco minutos.

3.6.1 Calibração de parâmetros

Os parâmetros do Algoritmo Memético foram calibrados por meio de testes A/B isolados, nos quais uma única configuração é alterada e comparada à configuração de referência, com as demais mantidas fixas, em execuções independentes de um mesmo conjunto de dados. A Tabela 3.4 resume os testes realizados.

Tabela 3.4: Testes de calibração dos parâmetros do Algoritmo Memético

Teste	Pergunta	Resultado	Decisão
Peso de capacidade	Qual valor de W_{cap} equilibra custo e violação?	Varredura de 200 a 5000 na instância média; 1900 teve o menor Z entre os pesos com violação média abaixo de 1 paciente-dia (Figura 3.1)	$W_{cap} = 1900$
Busca local nos elites	Refinar também os elites?	Z melhorou 2%, 23% e 45% nas instâncias pequena, média e grande	Adotado
População escalada	Aumentar a população com o porte?	Com 480 indivíduos, a instância grande completou 1 geração e Z quase dobrou em relação a 30 indivíduos	População fixa em 30
Perturbação	Reiniciar parte da população na estagnação?	Sem ganho consistente nas instâncias média e grande	Não adotado
Fração refinada	Refinar menos indivíduos para ter mais gerações?	23 gerações contra 14, mas Z 6% pior	Mantidos 30%
Relaxação linear	Usar a relaxação linear como limite inferior?	Não resolvida em 300 s na grande; construção de 289 s na muito grande	Limite do <i>branch-and-bound</i>

O peso da penalidade de capacidade W_{cap} (Equação 3.17) foi o parâmetro com a calibração mais detalhada. Tendo sido obtido por meio de uma varredura na instância média (conjunto 1). Foram testados dez valores, de 200 a 5000, com três execuções independentes de 60 segundos para cada um e os demais parâmetros mantidos fixos. Para cada execução, registraram-se a função objetivo Z e a violação de capacidade V_{cap} da melhor solução encontrada.

A Figura 3.1 mostra os dois efeitos opostos do peso. Com $W_{cap} = 200$, a penalidade é fraca o bastante para a busca aceitar exceder a capacidade em troca de um Z menor: obtém-se o menor Z médio (31 367), mas com 8,7 pacientes-dia sem leito. Ao aumentar o peso, a violação cai rapidamente, enquanto Z tende a crescer, já que a busca passa a priorizar a viabilidade.

Como critério de escolha, adotou-se o menor Z entre os pesos que mantêm a violação média abaixo de 1 paciente-dia. Três valores atendem a essa condição: 1900 ($Z = 37\,650$; $V_{cap} = 0,7$), 3300 ($Z = 41\,325$; $V_{cap} = 0,3$) e 5000 ($Z = 40\,992$; $V_{cap} = 0,7$), e por isso foi adotado $W_{cap} = 1900$.

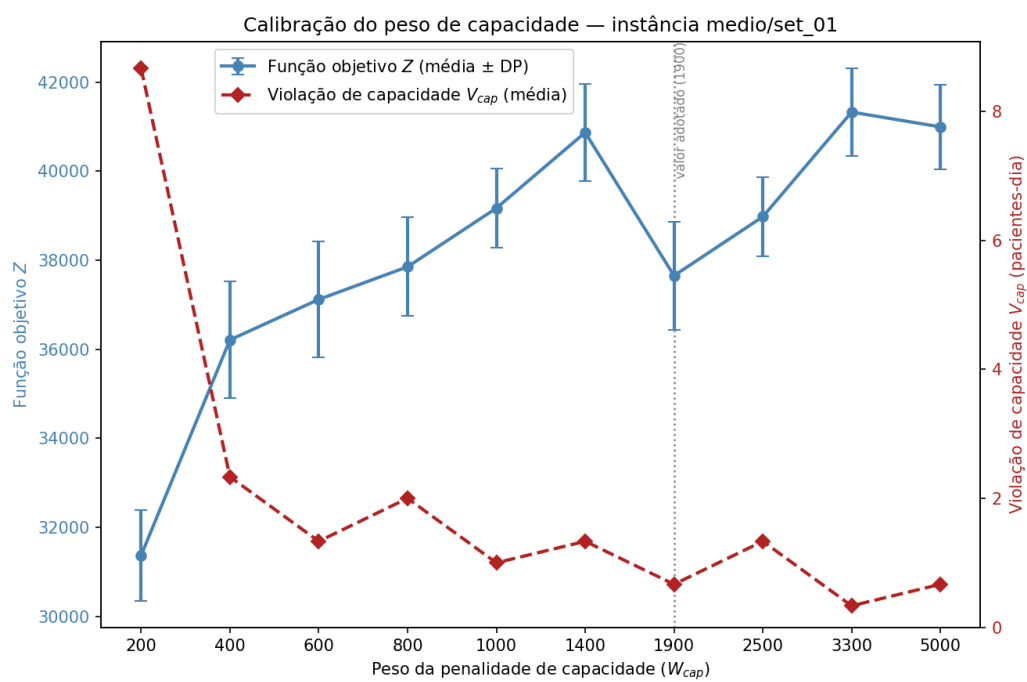


Figura 3.1: Função objetivo Z (média $\pm$ desvio padrão) e violação de capacidade V_{cap} (média) da melhor solução heurística para cada valor de W_{cap} — instância média, conjunto 1, três execuções de 60 s por valor

Capítulo 4

Resultados

Este capítulo apresenta os resultados da comparação entre as abordagens exata e heurística nos quatro portes de instância, seguindo o protocolo da Tabela 3.3. Primeiro é apresentada uma visão geral; em seguida, os resultados de cada porte; por fim, a viabilidade das soluções heurísticas e o esforço computacional. Em cada porte, é mostrado primeiro um conjunto representativo (conjunto 1) e, em seguida, o resultado agregado dos 10 conjuntos.

4.1 Visão geral

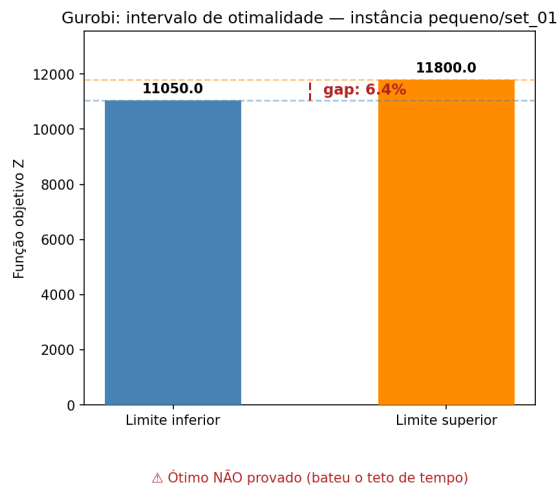
A Tabela 4.1 resume o valor da função objetivo Z (Equação 3.8) obtido por cada abordagem em cada porte.

Tabela 4.1: Função objetivo Z por porte (média entre os 10 conjuntos). Na heurística, Z da melhor execução de cada conjunto, sem a penalidade de capacidade

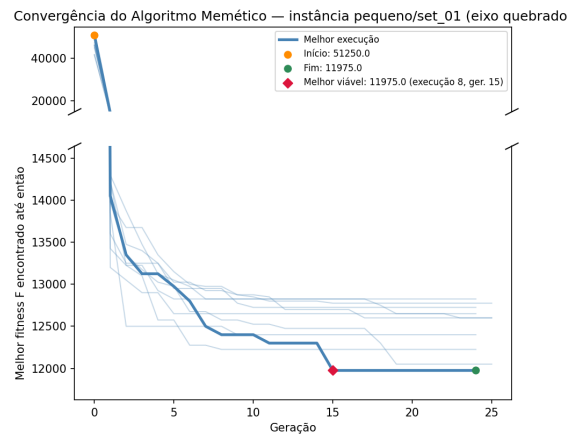
Porte	Gurobi (lim. sup.)	Gurobi (lim. inf.)	Gap	Heurística	Vitórias da heurística
Pequena	10 370,0	9 845,0	5,4%	10 567,5 $\pm$ 3 493,4	0/10
Média	83 992,5	23 950,0	71,4%	28 807,5 $\pm$ 5 560,8	10/10
Grande	820 270,0	7 735,0	99,1%	56 260,0 $\pm$ 9 935,3	10/10
Muito grande	3 277 170,0	0,0	100,0%	210 675,0 $\pm$ 22 841,7	10/10

4.2 Instância pequena

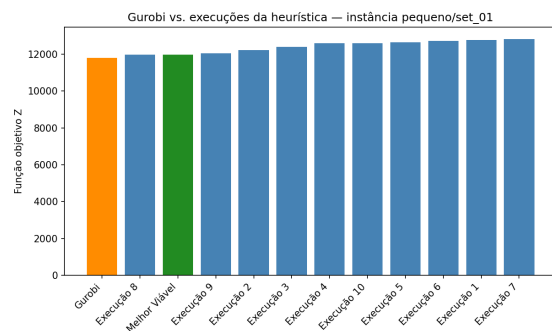
Resultados da instância pequena (100 pacientes, 30 s por execução).



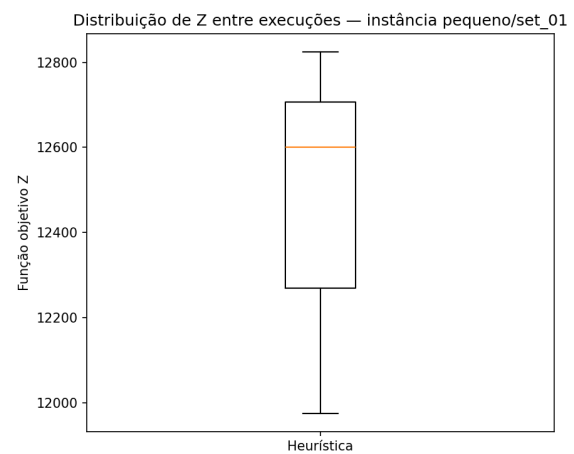
(a) Limites do Gurobi



(b) Convergência da heurística



(c) Gurobi vs. execuções da heurística



(d) Dispersão de Z entre execuções

Figura 4.1: Conjunto representativo (conjunto 1) — instância pequena

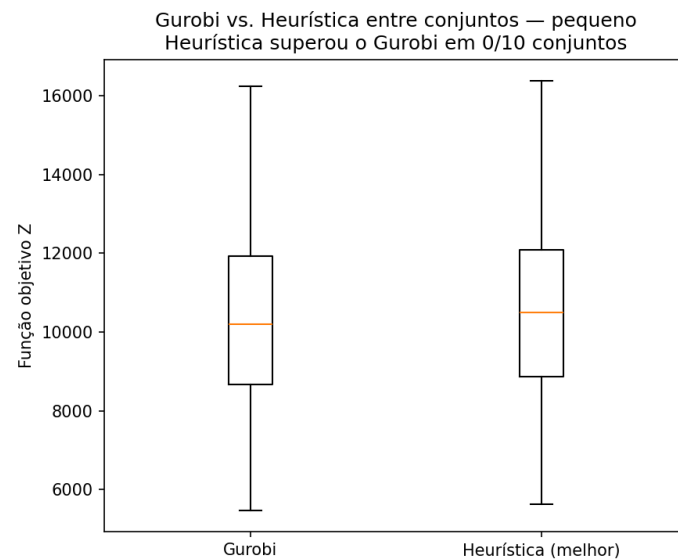


Figura 4.2: Z do Gurobi e da melhor solução heurística nos 10 conjuntos — instância pequena

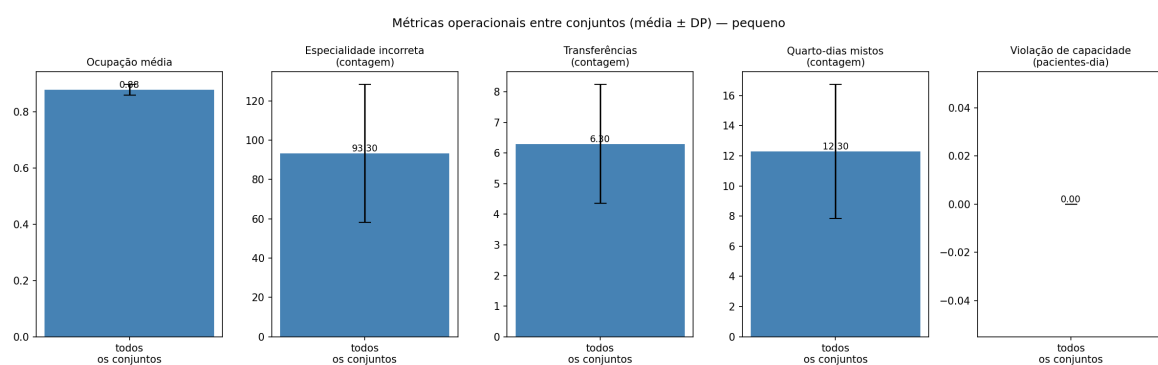
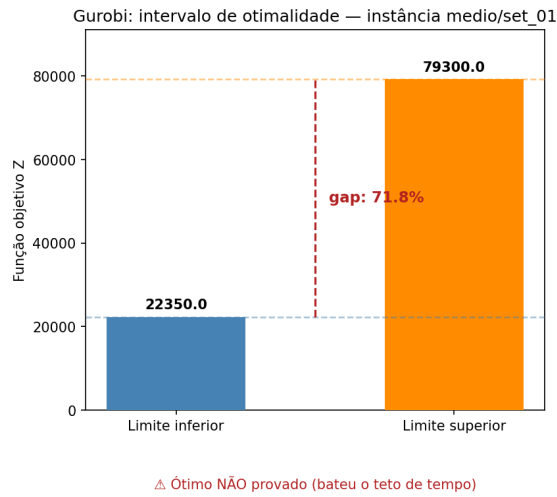


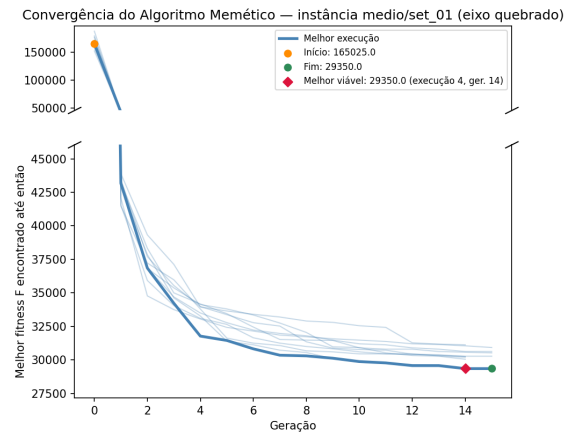
Figura 4.3: Métricas operacionais da melhor solução heurística (média $\pm$ desvio padrão entre conjuntos) — instância pequena

4.3 Instância média

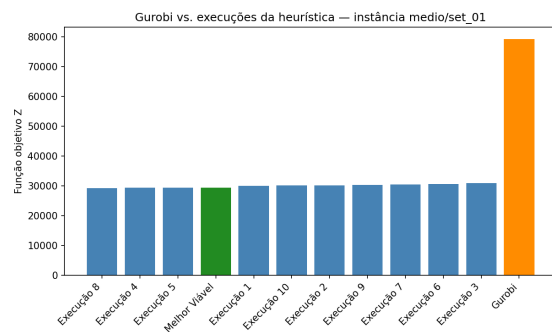
Resultados da instância média (400 pacientes, 90 s por execução).



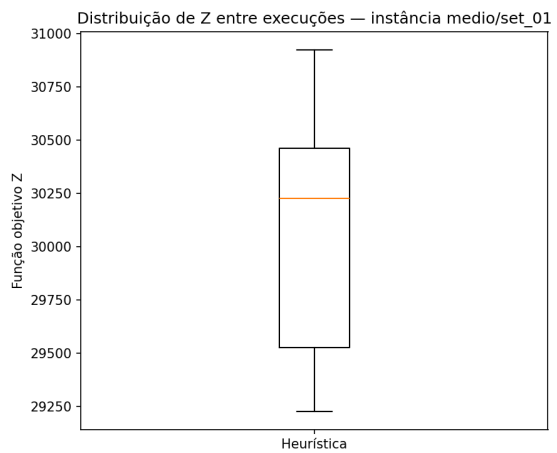
(a) Limites do Gurobi



(b) Convergência da heurística



(c) Gurobi vs. execuções da heurística



(d) Dispersão de Z entre execuções

Figura 4.4: Conjunto representativo (conjunto 1) — instância média

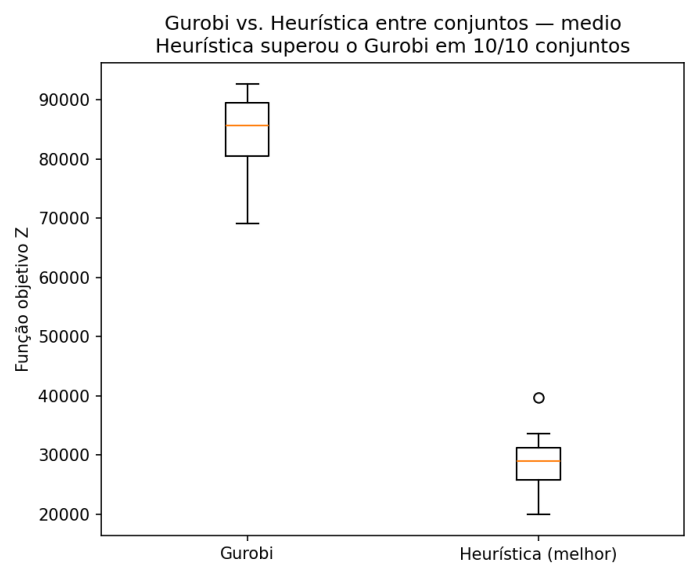


Figura 4.5: Z do Gurobi e da melhor solução heurística nos 10 conjuntos — instância média

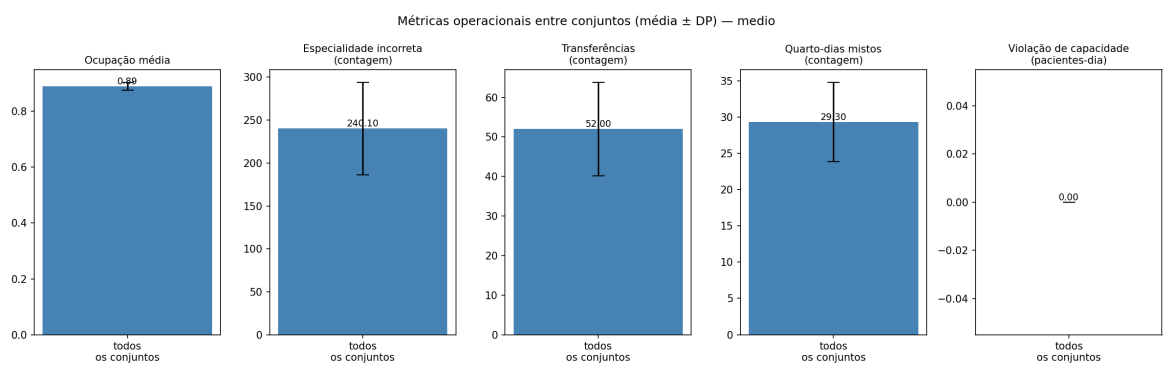
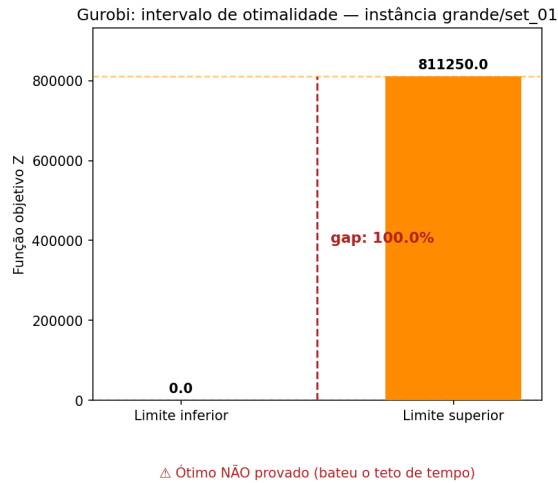


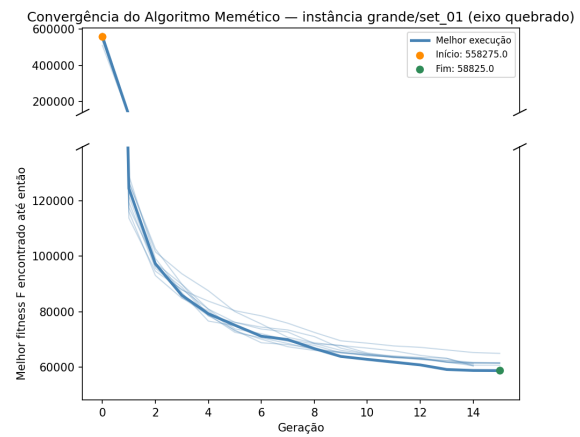
Figura 4.6: Métricas operacionais da melhor solução heurística (média $\pm$ desvio padrão entre conjuntos) — instância média

4.4 Instância grande

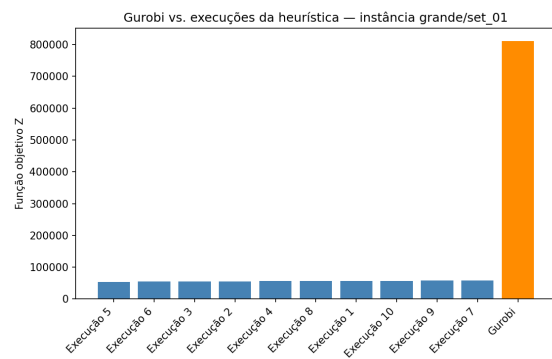
Resultados da instância grande (1600 pacientes, 360 s por execução).



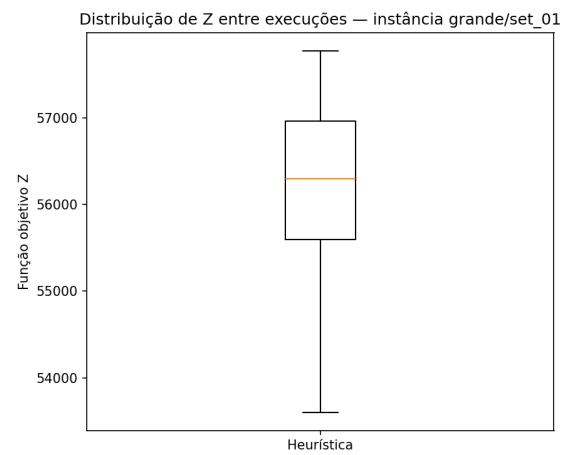
(a) Limites do Gurobi



(b) Convergência da heurística



(c) Gurobi vs. execuções da heurística



(d) Dispersão de Z entre execuções

Figura 4.7: Conjunto representativo (conjunto 1) — instância grande

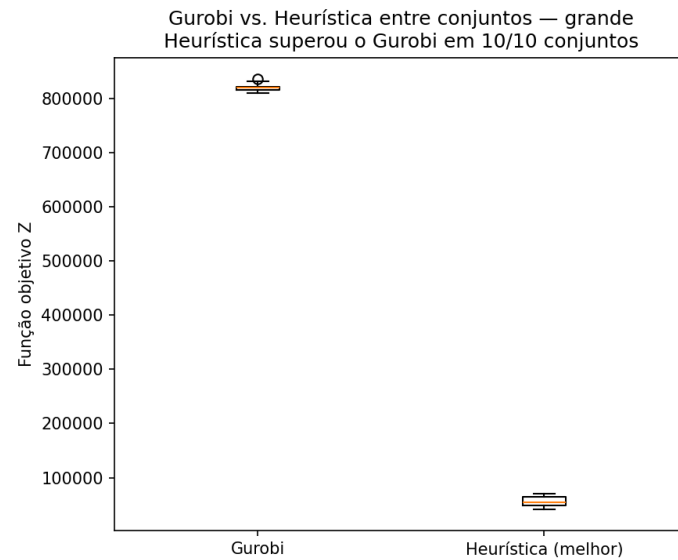


Figura 4.8: Z do Gurobi e da melhor solução heurística nos 10 conjuntos — instância grande

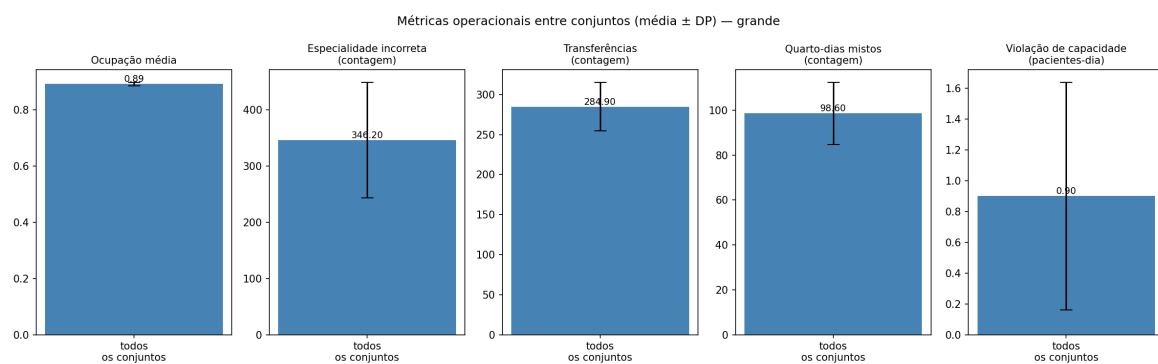


Figura 4.9: Métricas operacionais da melhor solução heurística (média $\pm$ desvio padrão entre conjuntos) — instância grande

4.5 Instância muito grande

Resultados da instância muito grande (6400 pacientes, 900 s por execução).

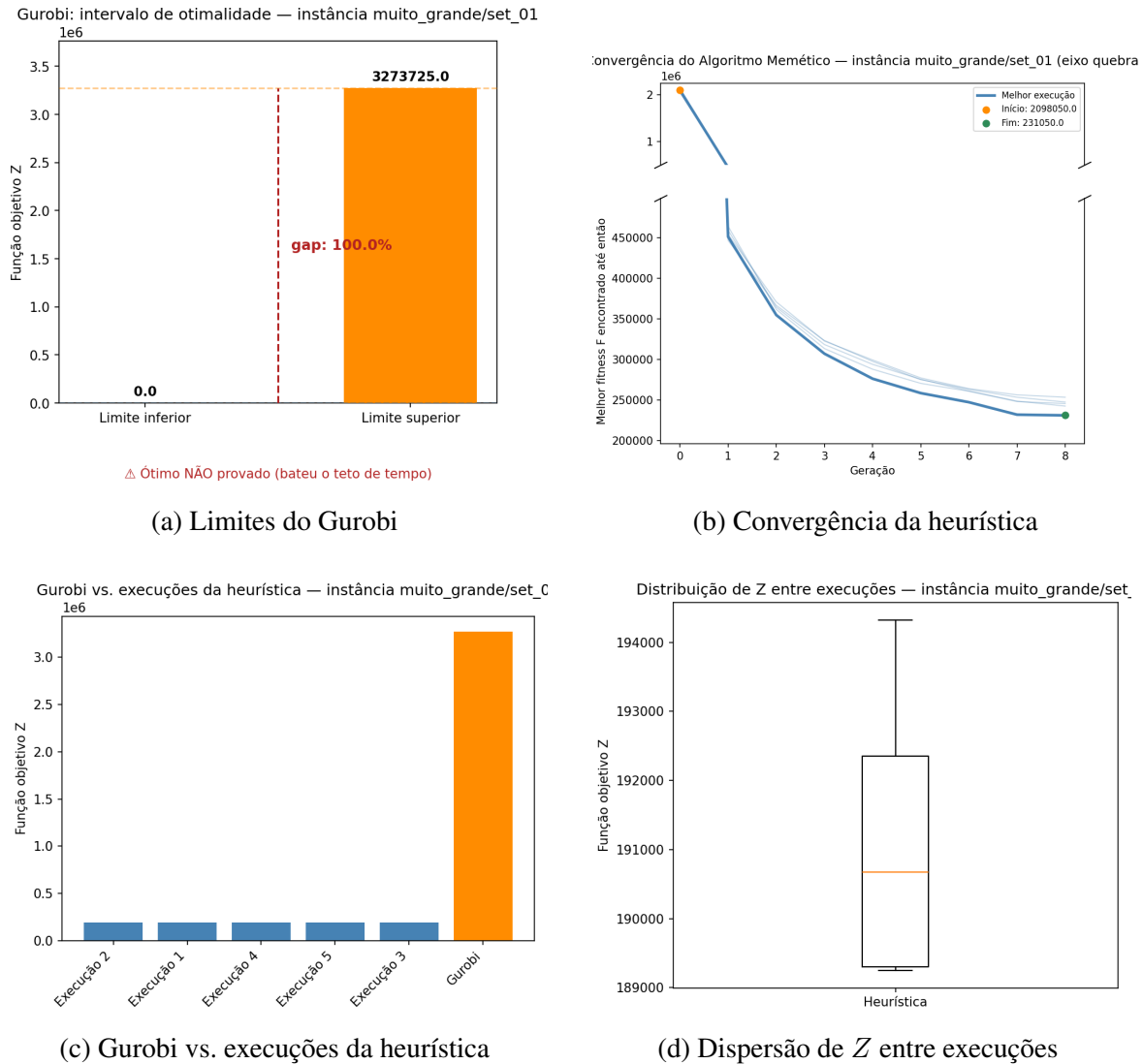


Figura 4.10: Conjunto representativo (conjunto 1) — instância muito grande

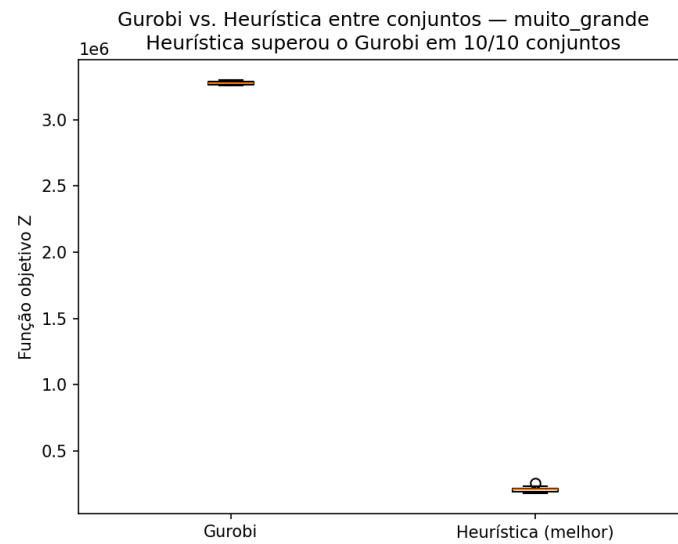


Figura 4.11: Z do Gurobi e da melhor solução heurística nos 10 conjuntos — instância muito grande

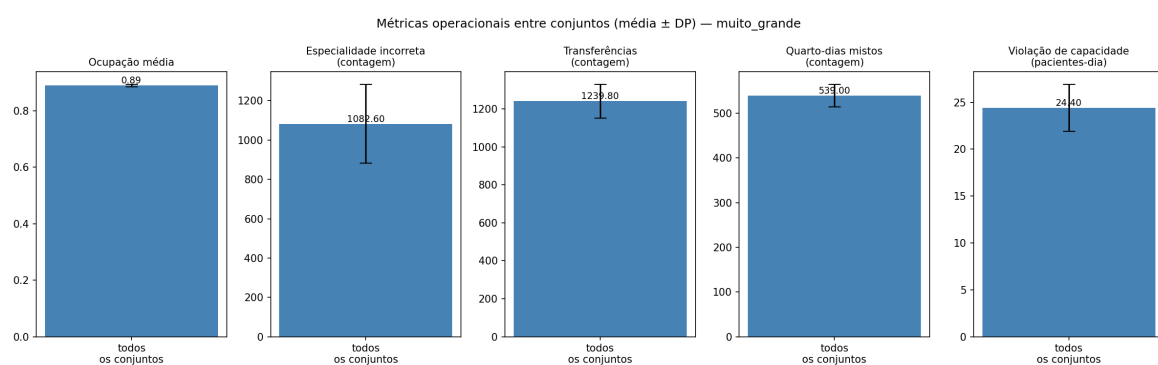


Figura 4.12: Métricas operacionais da melhor solução heurística (média $\pm$ desvio padrão entre conjuntos) — instância muito grande

4.6 Viabilidade das soluções heurísticas

A Tabela 4.2 mostra a violação de capacidade das soluções heurísticas, medida pela taxa ρ_{cap} (Equação 3.19).

Tabela 4.2: Viabilidade das soluções heurísticas por porte (melhor solução de cada conjunto, média $\pm$ desvio padrão entre os 10 conjuntos)

Porte	Execuções que encontraram solução viável	Nº Violações de capacidade (V_{cap})	Carga de internação (N_{pd})	Taxa de violação de capacidade ($\rho_{cap} = V_{cap}/N_{pd}$)
Pequena	100/100	$0,0 \pm 0,0$	$397,8 \pm 11,0$	$0,000\% \pm 0,000\%$
Média	100/100	$0,0 \pm 0,0$	$1\,611,6 \pm 21,1$	$0,000\% \pm 0,000\%$
Grande	5/100	$0,9 \pm 0,7$	$6\,420,6 \pm 41,7$	$0,014\% \pm 0,011\%$
Muito grande	0/50	$24,4 \pm 2,5$	$25\,601,0 \pm 107,7$	$0,095\% \pm 0,010\%$

4.7 Esforço computacional

A Tabela 4.3 compara o esforço de busca das duas abordagens dentro do mesmo teto de tempo.

Tabela 4.3: Esforço computacional por porte (média por execução)

Porte	Teto	Nº Iterações do Gurobi	Nº Avaliações da heurística	Nº Gerações
Pequena	30 s	1 112 503	2 202 174	25,5
Média	90 s	50 741	5 841 757	15,5
Grande	360 s	220 275	22 175 677	14,5
Muito grande	900 s	1 334	46 617 002	8,0

4.8 Síntese

Síntese dos resultados e ponte para a Conclusão.

Tabela 4.4: Resumo agregado entre os 10 conjuntos de cada porte (média $\pm$ desvio padrão)

Métrica	Pequena	Média	Grande	Muito grande
Instâncias				
Horizonte (dias)	15,1 $\pm$ 0,6	15,1 $\pm$ 0,3	15,0 $\pm$ 0,0	15,0 $\pm$ 0,0
Ocupação Média (todo o horizonte)	87,9% $\pm$ 1,9%	89,0% $\pm$ 1,4%	89,2% $\pm$ 0,6%	88,9% $\pm$ 0,4%
Pico de Ocupação Diária	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%
Mínimo de Ocupação Diária	11,3% $\pm$ 6,9%	14,3% $\pm$ 6,4%	12,3% $\pm$ 2,6%	11,0% $\pm$ 1,2%
Gurobi				
Z (Limite Superior)	10 370,0 $\pm$ 3 461,4	83 992,5 $\pm$ 7 816,0	820 270,0 $\pm$ 8 025,5	3 277 170,0 $\pm$ 14 301,5
Limite Inferior	9 845,0 $\pm$ 3 391,1	23 950,0 $\pm$ 5 336,1	7 735,0 $\pm$ 16 852,7	0,0 $\pm$ 0,0
Gap de Otimidade	5,4% $\pm$ 3,6%	71,4% $\pm$ 6,1%	99,1% $\pm$ 2,1%	100,0% $\pm$ 0,0%
Heurística				
Z	10 567,5 $\pm$ 3 493,4	28 807,5 $\pm$ 5 560,8	56 260,0 $\pm$ 9 935,3	210 675,0 $\pm$ 22 841,7
Métricas Operacionais				
Especialidade Incorreta (contagem)				
Gurobi	93,0 $\pm$ 35,7	369,9 $\pm$ 70,0	4 618,8 $\pm$ 56,6	18 470,5 $\pm$ 99,3
Heurística	93,3 $\pm$ 35,2	240,1 $\pm$ 54,0	346,2 $\pm$ 102,6	1 082,6 $\pm$ 200,2
Transferências (contagem)				
Gurobi	3,7 $\pm$ 1,8	575,4 $\pm$ 150,3	4 789,4 $\pm$ 39,9	19 170,7 $\pm$ 108,2
Heurística	6,3 $\pm$ 1,9	52,0 $\pm$ 11,8	284,9 $\pm$ 30,4	1 239,8 $\pm$ 88,2
Quarto-dias Mistos (contagem)				
Gurobi	11,8 $\pm$ 4,1	243,1 $\pm$ 38,8	1 585,6 $\pm$ 19,1	6 287,8 $\pm$ 29,7
Heurística	12,3 $\pm$ 4,4	29,3 $\pm$ 5,5	98,6 $\pm$ 13,8	539,0 $\pm$ 25,3
Violação de Capacidade (pacientes-dia)				
Gurobi	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0
Heurística	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0	0,9 $\pm$ 0,7	24,4 $\pm$ 2,5

Referências Bibliográficas

- [1] BRANDEAU, M. L.; SAINFORT, F.; PIERSKALLA, W. P. *Operations research and health care: a handbook of methods and applications*. [S.l.]: Springer Science & Business Media, 2004. v. 70.
- [2] SCHÄFER, F. et al. Combining machine learning and optimization for the operational patient-bed assignment problem. *Health Care Management Science*, v. 26, n. 4, p. 785, 2023.
- [3] HE, L. et al. A systematic review of research design and modeling techniques in inpatient bed management. *Computers & Industrial Engineering*, Elsevier, v. 127, p. 451–466, 2019.
- [4] CESCHIA, S.; SCHAERF, A. Modeling and solving the dynamic patient admission scheduling problem under uncertainty. *Artificial intelligence in medicine*, Elsevier, v. 56, n. 3, p. 199–205, 2012.
- [5] CESCHIA, S.; SCHAERF, A. Local search and lower bounds for the patient admission scheduling problem. *Computers & Operations Research*, Elsevier, v. 38, n. 10, p. 1452–1463, 2011.
- [6] DEMEESTER, P. et al. A hybrid tabu search algorithm for automatically assigning patients to beds. *Artificial Intelligence in Medicine*, Elsevier, v. 48, n. 1, p. 61–70, 2010.
- [7] TALBI, E.-G. *Metaheuristics: from design to implementation*. [S.l.]: John Wiley & Sons, 2009.
- [8] BELIËN, J.; DEMEULEMEESTER, E. Building cyclic master surgery schedules with leveled resulting bed occupancy. *European journal of operational research*, Elsevier, v. 176, n. 2, p. 1185–1204, 2007.
- [9] ALA, A.; CHEN, F. Appointment scheduling problem in complexity systems of the healthcare services: A comprehensive review. *Journal of Healthcare Engineering*, Wiley Online Library, v. 2022, n. 1, p. 5819813, 2022.
- [10] VISSERS, J. M.; ADAN, I. J.; DELLAERT, N. P. Developing a platform for comparison of hospital admission systems: An illustration. *European Journal of Operational Research*, Elsevier, v. 180, n. 3, p. 1290–1301, 2007.